

Backtracking

ఎంచుకో → లోతుకి వెళ్ళు → వెనక్కి తీసుకో. ఆ మూడో అడుగు ఈ pattern కి పేరు ఇచ్చింది, మరియు అదే అందరూ మర్చిపోయేది.

మూడు విరుపులు, మూడూ కొలిచినవి. మొదటిది – కత్తిరింపు లేకుండా N-Queens **1,741 రెట్లు** ఎక్కువ nodes తాకుతుంది. రెండోది – నకళ్ళు ఉన్న array కి 24 permutations వస్తాయి, అందులో **6 మాత్రమే** ప్రత్యేకమైనవి. మూడోది – ఫలితాన్ని copy చేయకపోతే **100% cases** lo అన్ని జవాబులూ ఖాళీ.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output, ప్రతి శాతం నిజంగా `node` lo run చేసినదే.

ఈ DOC LO విరువుల ఆకారం మిగతా వాటికంటే వేరు

Deep 01-06 lo సమస్య ఎప్పుడూ ఒక ఊహ: predicate ఏకదిశ, బరువులు ≥ 0 , memo key పూర్తి.

Backtracking ఏ ఊహ చేయదు – అది అన్నిటిని చూస్తుంది. కాబట్టి ఇక్కడ విరువులు వేరే రకం:

§4 – సరైన జవాబు, వేల రెట్లు ఎక్కువ పని.

§6 – సరైన జవాబులు, నాలుగు రెట్లు ఎక్కువ సార్లు.

§7 – JavaScript యొక్క reference semantics, మరియు 100% తప్పు.

అంటే backtracking lo మీ శత్రువు తప్పు తర్కం కాదు – వృథా.

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. Backtracking యొక్క మూడు అడుగులు
2. అడగాల్సిన ప్రశ్నలు

Part 2 – మొదటి విరువు: ఎప్పుడు కత్తిరించాలి

3. Step – N-Queens · మూడు స్థాయిల కత్తిరింపు
4. మొదటి విరువు – 9,60,800 nodes vs 552

Part 3 – రెండో విరువు: నకళ్ళు

5. Step – నకళ్ళు ఉన్న array
6. రెండో విరువు – 24 ఫలితాలు, 6 ప్రత్యేకమైనవి
7. Step – ఒక్క పంక్తి · మరియు అది ఎందుకు పనిచేస్తుంది

Part 4 – మూడో విరువు: reference

8. మూడో విరువు – అన్ని జవాబులూ ఖాళీ
9. Step – copy ఎక్కడ చేయాలి

Part 5 – నిరూపణ మరియు ప్రదర్శన

10. మొత్తం code
11. 24,000 cases · mutation testing
12. ఒక నియమం లేకపోవడం – "చెల్లుతుంది" vs "అన్నీ ఉన్నాయి"
13. బతికిన రెండు మార్పులు

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. Backtracking యొక్క మూడు అడుగులు

```
for (const choice of options) {
  make(choice);      // 1. ఎంచుకో
  recurse();          // 2. లోతుకి వెళ్ళు
  undo(choice);       // 3. వెనక్కి తీసుకో ← ఇదే పేరు
}
```

ఆ మూడో అడుగు ఎందుకు ఉంది

మీరు ఒకే స్థితిని (path , used , board) పదే పదే వాడుతున్నారు – ప్రతి శాఖకీ కొత్తది తయారు చేయడం లేదు.

ఎందుకంటే ప్రతి శాఖకీ కొత్త copy తయారు చేస్తే **memory ఘాతీయంగా** పెరుగుతుంది.

కాబట్టి ఒప్పందం ఇది: "నేను ఈ స్థితిని అరుపు తీసుకుంటాను, మరియు వెళ్ళేటప్పుడు ఉన్నట్టే ఉంచేస్తాను."

ఆ ఒప్పందం విరిగితే – §11 లో `used[i] = 0` తీసేస్తే **permutations 6 కి బదులు 1**.

2. అడగాల్సిన ప్రశ్నలు

ప్రశ్న	ఎందుకు అడుగుతున్నాం
Input లో సకళ్ళు ఉన్నాయా?	§6 – ఉంటే ఫలితాలు 4 రెట్లు ఎక్కువ, అన్నీ ఒకటే
అన్ని జవాబులు కావాలా, ఒక్కటి చాలా?	ఒక్కటి చాలంటే మొదటిది దొరకగానే ఆగొచ్చు – చాలా వేగం
సంఖ్య కావాలా, జాబితా కావాలా?	సంఖ్య చాలంటే DP సరిపోవచ్చు – backtracking అనవసరం
ఒక ఎంపికని ఎన్నిసార్లు వాడొచ్చు?	ఒకసారే vs అపరిమితం – <code>go(k)</code> vs <code>go(k+1)</code> (§11)
ఎంత ముందుగా చెల్లదని చెప్పగలం?	ఇదే §4. ముందుగా చెప్పగలిగితే వేల రెట్లు ఆదా
n ఎంత?	Backtracking ఘాతీయం – $n \leq 20$ దాటితే ఆలోచించాలి

ఒక ప్రశ్న అడిగితే మరు దీన్ని నిజంగా నడిపారని తెలుస్తుంది

"ఒక పాక్షిక పరిష్కారం చెల్లదని నేను ఎంత ముందుగా చెప్పగలను?"

అదే backtracking యొక్క మొత్తం ఆట. §4 lo కొలుస్తాం:

చివర్ల తనిఖీ → 9,60,800 nodes.

ప్రతి అడుగులోనూ తనిఖీ → 552.

అదే code, అదే జవాబులు, **1,741 రెట్ల** తేడా – కేవలం "ఎప్పుడు అడిగాం" అనేదాని వల్ల.

Part 2 – మొదటి విరుపు: ఎప్పుడు కత్తిరించాలి

3. Step – N-Queens • మూడు స్థాయిల కత్తిరింపు

```
// (అ) కత్తిరింపు లేదు – అన్ని  $n^n$  అమరికలు తయారుచేసి చివర్లో తనిఖీ
const go = r => { if (r === n) { if (valid(pos)) count++; return; }
  for (let c = 0; c < n; c++) { pos[r] = c; go(r+1); } };

// (ఆ) నిలుపు వరుస మాత్రమే – permutations ( $n!$  అమరికలు)
if (!used[c]) { used[c]=1; pos[r]=c; go(r+1); used[c]=0; }

// (ఇ) ప్రతి అడుగులోనూ పూర్తి తనిఖీ – నిజమైన backtracking
if (ok(r,c)) { pos[r] = c; go(r+1); }
```

మూడూ అవే జవాబులు ఇస్తాయి. తేడా ఎన్ని దారులు నడిచాం అనేది.

4. మొదటి విరుపు – 9,60,800 nodes vs 552

N-Queens • శోధన వృక్షంలో ఎన్ని nodes ని తాళాం?

n	జవాబులు	కత్తిరింపు లేదు	నిలుపు వరుస మాత్రమే	ప్రతి అడుగులోనూ	ఎన్ని రెట్లు తక్కువ
4	2	341	65	17	
20×	5	10	3,906	326	54
72×	6	4	55,987	1,957	153
366×	7	40	9,60,800	13,700	552
1741×	8	92	(చాలా ఎక్కువ)	1,09,601	2,057
–					

N ని 4 నుంచి 7 కి పెంచితే - లాభం $20 \times$ నుంచి $1,741 \times$ కి

ఇది ఒక స్థిరమైన లాభం కాదు; అది **n తో పెరుగుతుంది.**

ఎందుకంటే ప్రతి కత్తిరింపు ఒక **ఉపవృక్షం మొత్తాన్ని** తొలగిస్తుంది. వృక్షం లోతు ఎక్కువైన కొద్దీ ఒక్కో కత్తిరింపు ఆదా చేసేది ఘాతీయంగా పెరుగుతుంది.

కాబట్టి "ఎంత ముందుగా కత్తిరించాం" అనేది n పెరిగే కొద్దీ ఎక్కువ ముఖ్యమవుతుంది.

మధ్య నిలువు వరుస - సగం ఆలోచన కూడా చాలా ఆదా చేస్తుంది

"రెండు queens ఒకే నిలువు వరుసలో ఉండకూడదు" అనేది ఒక్క నియమం. అది 9,60,800 ని **13,700** కి తెచ్చింది - 70 రెట్లు.

ఆపై వికర్ణ నియమం జోడిస్తే $13,700 \rightarrow$ **552** - ఇంకో 25 రెట్లు.

ప్రతి అదనపు constraint ఒక గుణకం. అవి కూడబడవు, గుణించబడతాయి.

Part 3 – రెండో విరుపు: నకళ్ళు

5. Step – నకళ్ళు ఉన్న array

[1, 2, 3] యొక్క permutations – 6, అన్నీ వేరువేరు. సులభం.

[1, 1, 2] ? Code ప్రకారం ఇప్పటికీ 6 – ఎందుకంటే code కి స్థానాలు తెలుసు, విలువలు కాదు. మొదటి 1 ని ముందు పెట్టడం మరియు రెండో 1 ని ముందు పెట్టడం – code కి అవి రెండు వేర్వేరు దారులు.

మనకి అవి ఒకటే.

6. రెండో విరుపు – 24 ఫలితాలు, 6 ప్రత్యేకమైనవి

నకళ్ళు ఉన్న array యొక్క permutations

array	naive ఇచ్చినవి	అందులో ప్రత్యేకమైనవి	వృథా	dedup ఇచ్చినవి
[1,2,3]	6	6	0	6 ✓
[1,1,2]	6	3	3	3 ✓
[1,1,1]	6	1	5	1 ✓
[1,1,2,2]	24	6	18	6 ✓
[1,1,1,2]	24	4	20	4 ✓

[1,1,1,2] → 24 ఫలితాలు, అందులో 4 ప్రత్యేకమైనవి • 83% వృథా

[1,1,1] ఇంకా ఘోరం: 6 ఫలితాలు, అన్నీ ఒకటే.

మరియు ఇది కేవలం వృథా కాదు – ఫలితం తప్పు. "ప్రత్యేకమైన permutations ఇవ్వు" అని అడిగితే నకళ్ళతో కూడిన జాబితా సరైన జవాబు కాదు.

మరియు ఆ వృథా ఘాతీయం: n మూలకాల్లో అన్నీ ఒకటే అయితే n! ఫలితాలు, అందులో 1 ప్రత్యేకమైనది.

7. Step – ఒక్క పంక్తి • మరియు అది ఎందుకు పనిచేస్తుంది

```
const a = [...arr].sort((x, y) => x - y); // ఒకేలాంటివి పక్కపక్కన
...
if (i > 0 && a[i] === a[i-1] && !used[i-1]) continue;
```

ఆ పంక్తి చెప్పేది ఒక్క వాక్యంలో

"ఒకే విలువ ఉన్న వాటిలో, ఎడమవైపుది వాడకుండా కుడివైపుది వాడకూడదు."

అంటే `[1a, 1b, 2]` లో `1b` ని మొదట వాడటం నిషేధం. కాబట్టి `1a` ముందు, `1b` తర్వాత – ఒకే ఒక క్రమం.

ఫలితం: ఒకేలా కనిపించే అన్ని దారుల్లో సరిగ్గా ఒక్కటే అనుమతించబడుతుంది. నకళ్ళు తయారై, ఆపై filter చేయబడవు – అవి అసలు తయారే కావు.

మరియు `sort` లేకపోతే ఈ పంక్తి పనిచేయదు – నకళ్ళు పక్కపక్కన లేకపోతే `a[i] == a[i-1]` ఎప్పుడూ నిజం కాదు.

Part 4 – మూడో విరుపు: reference

8. మూడో విరుపు – అన్ని జవాబులూ ఖాళీ

```
res.push(path);          // ← copy లేదు  
res.push([...path]);      // ← copy ఉంది
```

```
a = [1,2]  
copy చేస్తే : [[],[2],[1],[1,2]]  
copy లేకపోతే : [[],[],[],[1]]  
  
a = [1,2,3]  
copy చేస్తే : [[],[3],[2],[2,3],[1],[1,3],[1,2],[1,2,3]]  
copy లేకపోతే : [[],[],[],[1],[1],[1],[1],[1]]
```

20,000 యాదృచ్ఛిక cases lo copy లేకపోతే తప్పు జవాబు: 20,000 (100.0%)

ఫలితాల సంఖ్య సరైనది (4, 8) • విషయం అంతా ఖాళీ

`res.push(path)` అనేది ఆ array ని **copy చేయదు** – అది ఆ array కి ఒక **సూచన** (reference) మాత్రమే పెడుతుంది.

Backtracking ఆ ఒకే array ని పదే పదే మారుస్తుంది. Recursion ముగిసేసరికి **అన్ని pop()** లూ జరిగిపోయాయి, కాబట్టి array ఖాళీ.

`res` lo ఉన్న 8 entries – **అన్నీ ఆ ఒక్క ఖాళీ array నే.**

మరియు DSA Deep 03 §8 యొక్క అదే JavaScript సమస్య: Set reference ప్రకారం పోల్చినట్టే, push reference ప్రకారం నిల్వ చేస్తుంది.

9. Step – copy ఎక్కడ చేయాలి

నియమం: ఫలితంలో పెట్టేటప్పుడు **COPY**. ఇంకెక్కడా కాదు.

ఫలితంలో పెట్టేటప్పుడు → `res.push(...path)` . తప్పనిసరి.

Recursion కి ఇచ్చేటప్పుడు → copy వద్దు. అదే backtracking యొక్క మొత్తం ఉద్దేశం – ఒకే స్థితిని పంచుకోవడం.

ప్రతి recursive call కీ copy చేస్తే memory $O(\text{శాఖల సంఖ్య} \times \text{లోతు})$ అవుతుంది – మరియు అప్పుడు మీరు backtracking రాయడం లేదు, మీరు కేవలం **ఖరీదైన recursion** రాస్తున్నారు.

Part 5 – నిరూపణ మరియు ప్రదర్శన

10. మొత్తం code

```
function permutations(arr) {
  const a = [...arr].sort((x, y) => x - y); // ఒకేలాంటివి పక్కపక్కన
  const res = [], path = [], used = new Uint8Array(a.length);
  const go = () => {
    if (path.length === a.length) { res.push([...path]); return; } // copy (§9)
    for (let i = 0; i < a.length; i++) {
      if (used[i]) continue;
      if (i > 0 && a[i] === a[i-1] && !used[i-1]) continue; // నకలు దాటవేత
      (§7)
      used[i] = 1; path.push(a[i]);
      go();
      path.pop(); used[i] = 0; // ← వెనక్కి తీసుకోవడం
    }
  };
  go();
  return res;
}

function nQueens(n) {
  const res = [], pos = new Array(n);
  const ok = (r, c) => {
    for (let i = 0; i < r; i++)
      if (pos[i] === c || Math.abs(pos[i] - c) === r - i) return false;
    return true;
  };
  const go = r => {
    if (r === n) { res.push([...pos]); return; }
    for (let c = 0; c < n; c++) if (ok(r, c)) { pos[r] = c; go(r + 1); }
  };
  go(0);
  return res;
}
```

11. 24,000 cases • mutation testing

పది నియమాలు: **permutations** lo నకళ్ళు లేవు, ప్రతి ఫలితమూ అసలు మూలకాల అమరిక, సంఖ్య = $n!$ / $\prod(\text{నకలు!})$, **subsets** ఖచ్చితంగా 2^n , ప్రతి ఉపసమితి ఒక subsequence, **nQueens** తెలిసిన సంఖ్యలతో సరిపోవాలి, ప్రతి **queens** అమరికా నిజంగా చెల్లుతుంది, ప్రతి **combination** మొత్తం = target, **brute force** కనుగొన్న ప్రతి **combination** ఇక్కడా ఉంది, **combinations** lo నకళ్ళు లేవు.

300 runs · 9,000 permutation cases · 9,000 subset cases · 6,000 combos

నియమ ఉల్లంఘనలు: 0

ఏ దారులు నడిచాయి:

perms	9,000
subsets	9,000
queens	300
combos	6,000
withDupes	4,596
emptyRes	70

==== mutation testing ====

మార్పు లేని code	→ 0/300 విఫలం
ఫలితంలో [...path] కి బదులు path నే	→ 120/120 విఫలం
ఉదా: permutations([0,1,2]) lo నకళ్ళు	
permutations lo నకలు-దాటవేత పంక్తి తీసేస్తే	→ 120/120 విఫలం
ఉదా: permutations([0,1,2,1,2]) lo నకళ్ళు	
permutations lo '!used[i-1]' ని 'used[i-1]' గా	→ 0/120 విఫలం
permutations lo 'used[i] = 0' (వెనక్కి తీసుకోవడం) తీసేస్తే	→ 120/120 విఫలం
ఉదా: permutations([0,1,2]) = 1, ఆశించినది 6	
subsets lo path.pop() తీసేస్తే	→ 120/120 విఫలం
ఉదా: subsets → [5,4] subsequence కాదు	
nQueens lo వికర్ణ తనిఖీ తీసేస్తే	→ 86/120 విఫలం
ఉదా: nQueens(2) = 2, ఆశించినది 0	
combinationSum lo go(k, ...) ని go(k+1, ...) గా	→ 120/120 విఫలం
ఉదా: combinationSum([1,2,1,1], 10) lo [1,1,1,1,1,1,1,1]	
combinationSum lo 'break' ని 'continue' గా	→ 0/120 విఫలం

used[i] = 0 తీసేస్తే **permutations 6 కి బదులు 1** – §1 యొక్క ఒప్పందం విరిగితే అదే జరుగుతుంది. మొదటి దారి నడిచాక ప్రతి used[i] ఇంకా 1 గానే ఉంటుంది, కాబట్టి రెండో దారి **ఎప్పుడూ మొదలవదు**.

12. ఒక నియమం లేకపోవడం – "చెల్లుతుంది" vs "అన్నీ ఉన్నాయి"

combinationSum lo go(k, ...) ని go(k+1, ...) గా మార్చడం – అంటే "ప్రతి మూలకం ఎన్నిసార్లైనా" నుంచి "ఒకసారే" కి. మొదట అది **0/120**.

నా తొమ్మిది నియమాలూ ఒకే దిశలో చూస్తున్నాయి

అవన్నీ ఇలా అడుగుతున్నాయి: **"ఈ ఫలితం చెల్లుతుందా?"** – మొత్తం target ఏనా, మూలకాలు అభ్యర్థుల్లోనివేనా, నకళ్ళు లేవా.

go(k+1) అనే mutation **తక్కువ** ఫలితాలు ఇస్తుంది, మరియు ఇచ్చిన ప్రతి ఒక్కటే **పూర్తిగా చెల్లుబాటు**. కాబట్టి ఒక్క నియమమూ విఫలం కాలేదు.

లోపించినది: **"అన్ని చెల్లుబాటు ఫలితాలూ ఉన్నాయా?"**

```
// నియమం 9 - *పూర్తిత్వం*: brute force కనుగొన్న ప్రతి కలయికా ఇక్కడ ఉండాలి
const truth = new Set(); bruteForce(0, target, []);
for (const t of truth) if (!got.has(t))
    bad.push(`combinationSum([${c}], ${target}) lo [${t}] తప్పిపోయింది`);
```

ఆ ఒక్క నియమం జోడించాక – **120/120**, మరియు ఉదాహరణ `[1,1,1,1,1,1,1,1,1]` తప్పిపోయింది.

ఈ **SERIES LO** పదే పదే వచ్చిన జంట, ఇక్కడ మళ్ళీ

HLD docs lo: "ఏదే తప్పు జరగలేదు" నియమాలకి ఒక "ఏదే జరగాలి" జోడీ కావాలి.

ఇక్కడ: "ప్రతి ఫలితమూ సరైనది" కి ఒక "ప్రతి సరైనదే ఫలితంలో ఉంది" జోడీ కావాలి.

మొదటిది soundness. రెండోది completeness. ఒకటి లేకుండా రెండోది అర్థరహితం – ఖాళీ జాబితా ఎప్పుడూ "సరైనది", మరియు ఎప్పుడూ పనికిరానిది.

13. బతికిన రెండు మార్పులు

రెండూ సరైనతనాన్ని మార్చవు – అవి పనిని మారుస్తాయి. కొలిచాను:

array	!	used[i-1]	nodes	used[i-1]	nodes	ఫలితాలు సమానమా
[1,1,2]	]		9	12		అవును
[1,1,1,2]	]		14	32		అవును
[1,1,2,2]	]		19	33		అవును
[1,1,1,1,2]	]		20	96		అవును
[1,1,2,2,3]	]		90	150		అవును

!USED[I-1] VS USED[I-1] – అవే ఫలితాలు, 4.8 రెట్లు ఎక్కువ పని

రెండు రూపాలూ ఒక్కే ప్రత్యేక permutation కి **ఒక్క దారే** అనుమతిస్తాయి – కాబట్టి రెండూ సరైనవి.

తేడా: `!used[i-1]` ఆ దారిని **ముందుగానే** కత్తిరిస్తుంది (ఎడమవైపుది ఇంకా వాడకపోతే వెంటనే ఆపు).

`used[i-1]` లోతుకి వెళ్ళి ఆపై తిరస్కరిస్తుంది.

`[1,1,1,1,2]` కి **20 nodes vs 96**. §4 యొక్క అదే పాఠం – **ఎంత ముందుగా కత్తిరించాం** అనేదే మొత్తం.

రెండోది `break` → `continue` : sorted కాబట్టి మిగతావన్నీ కూడా పెద్దవే, కానీ `continue` వాటిని **ఒక్కొక్కటిగా** తనిఖీ చేస్తుంది. మళ్ళీ – సరైన జవాబు, ఎక్కువ పని.

14. Follow-ups – memoization, iterative, constraint propagation

"Backtracking కి memo జేడించొచ్చా?"

కొన్నిసార్లు – మరియు అప్పుడు అది **DP అవుతుంది** (Deep 04). షరతు: ఉప-సమస్యలు **పునరావృతం కావాలి**, మరియు వాటిని ఒక key తో సూచించగలగాలి. N-Queens lo ప్రతి పాక్షిక board ప్రత్యేకమైనది కాబట్టి memo పనికిరాదు. కానీ "n మెట్లు ఎక్కడానికి ఎన్ని దారులు" lo స్థితి కేవలం **n** – అక్కడ backtracking వృథా, DP సరైనది.

"Recursion కి బదులు iterative రాయొచ్చా?"

అవును – ఒక explicit stack తో. Deep 04 §9 lo కొలిచిన **~9,181 లోతు పరిమితి** ఇక్కడా వర్తిస్తుంది, కాబట్టి లోతు ఎక్కువైతే ఇది అవసరం. కానీ backtracking lo లోతు సాధారణంగా n (small), మరియు **వెడల్పు** ఘాతీయం – కాబట్టి stack overflow అరుదు. **నేను iterative version కొలవలేదు.**

"Sudoku లాంటి వాటికి ఇంకా వేగం?"

Constraint propagation: ఒక గడిలో ఒకే ఒక అవకాశం మిగిలితే, recursion అవసరం లేకుండానే దాన్ని నింపడం. మరియు **అతి తక్కువ అవకాశాలు ఉన్న గడిని ముందు** ఎంచుకోవడం (most-constrained-variable) – ఇది §4 యొక్క ఆలోచనని ఇంకొక అడుగు ముందుకి తీసుకెళ్తుంది: **కత్తిరించడమే కాదు, ఏ శాఖని ముందు చూడాలో కూడా ఎంచుకోవడం. నేను వీటిని కొలవలేదు.**

"ఈ doc lo నేను ఏమి కొలవలేదు"

మూడు విషయాలు: **constraint propagation** (ఫైన), **iterative backtracking**, మరియు **bitmask తో N-Queens** – నిలువు వరుసలు/వికర్ణాలని మూడు bitmasks గా ఉంచితే $ok(r, c)$ $O(n)$ నుంచి **$O(1)$** అవుతుంది; nodes సంఖ్య మారదు కానీ ఒక్కో node ఖర్చు తగ్గుతుంది, మరియు ఆ తేడాని నేను కొలవలేదు.

15. ఏమి నేర్చుకున్నాం

1. **మూడో అడుగే పేరు.** `undo()` తీసేస్తే permutations **6 కి బదులు 1** – ఎందుకంటే రెండో దారి ఎప్పుడూ మొదలవదు.

2. **కత్తిరింపు లాభం n తో పెరుగుతుంది.** $n = 4$ కి $20\times$, $n = 7$ కి **$1,741\times$** – ఎందుకంటే ప్రతి కత్తిరింపు ఒక ఉపవృక్షం మొత్తాన్ని తొలగిస్తుంది.

3. **Constraints కూడబడవు, గుణించబడతాయి.** నిలువు వరుస నియమం $70\times$, ఆపై వికర్ణ నియమం ఇంకో $25\times$.

4. **నకళ్ళు వృథా మాత్రమే కాదు – తప్పు జవాబు.** `[1, 1, 1, 2]` $\rightarrow$ 24 ఫలితాలు, **4 ప్రత్యేకమైనవి.**

5. **నకలు దాటవేత filter కాదు, నివారణ.** ఆ ఒక్క పంక్తి నకళ్ళని తయారవకుండా ఆపుతుంది.

6. `res.push(path)` – **100% తప్పు, మరియు సంఖ్య సరైనదే.** JavaScript reference ని నిల్వ చేస్తుంది, మరియు recursion ముగిసేసరికి ఆ array ఖాళీ.

7. "ప్రతి ఫలితమూ సరైనది" అనేది సగం పరీక్ష. ఖాళీ జాబితా ఎప్పుడూ సరైనది. **Completeness** నియమం జోడించాక ఒక mutation 0/120 → **120/120**.

8. కొన్ని bugs జవాబుని మార్చవు, పనిని మారుస్తాయి. `used[i-1]` రూపం **4.8** రెట్లు ఎక్కువ nodes, అదే ఫలితాలు.

ఈ **DOC** నుంచి ఒక్క వాక్యం గుర్తుపెట్టుకోవాలంటే

Backtracking lo మీరు రాసే code అంతా "ఎలా వెతకాలి" గురించి కాదు – "ఎప్పుడు ఆపాలి" గురించి.

వెతకడం అల్పమైనది: అన్ని ఎంపికలూ ప్రయత్నించు. ఆ భాగం **మూడు** పంక్తులు.

మిగతాదంతా ఆపడం గురించి – ఈ శాఖ చెల్లదు (§4), ఈ శాఖ ఇంతకుముందే చూశాం (§7), ఇక్కడ నుంచి ఏదీ సరిపోదు (§13).

మరియు ప్రతి "ఆపు" ఒక ఉపవృత్తం మొత్తాన్ని తొలగిస్తుంది. **అందుకే ఒక్క పంక్తి 1,741 రెట్లు విలువైనది.**